Annexes

Convention de nommage des commits

Vous devez respecter la convention de nommage des commits décrite ci-dessous. Elle est inspirée de la convention de nommage utilisée pour le projet Angular.

Les messages de vos commits doivent respecter la structure suivante :

```
<type>(<scope>): <short summary>
  |       |             |
  |       |             └⫸ Résumé de ce que vous avez mis dans le commit.
  |       |
  |       └⫸ Commit Scope: le nom de la fonctionnalité concernée par ce
commit | rien.
  |
  └⫸ Commit Type: build|docs|feat|fix|perf|refactor
```

`<type>` doit prendre une des valeurs suivantes :

- `build` : changements qui affectent les options du projet ou les dépendances externes (librairies)
- `docs` : changements qui concernent la documentation (analyse, readme, ...)
- `feat` : développement d'une nouvelle fonctionnalité
- `fix` : correction d'un bug
- `perf` : modification qui améliorer la performance
- `refactor` : modification qui se contente de restructurer le code sans corriger un bug ni apporter une nouvelle fonctionnalité

`<scope>` doit être utilisé dans le cas où les changements concernent une fonctionnalité principale (avec d'éventuels impacts sur d'autres fonctionnalités). Si le changement n'est pas lié à une fonctionnalité, mais est plus général, le scope (et les parenthèses) peut être omis.

Évaluation

Chaque itération sera évaluée et interviendra dans la note finale du projet. Nous vous demandons donc un travail régulier.

Le travail doit être réparti de manière **équitable sur l'ensemble des membres de l'équipe de projet**. Ceci sera vérifié, notamment, sur base de :

- la présence et la participation active pendant les cours ;
- la répartition des commits sur les différents utilisateurs (nombre de commits et leur répartition dans le temps) ; il est donc fondamental que vos commits vous soient attribués et pour cela n'oubliez pas de définir votre nom et votre email dans Git. Si vous travaillez en appliquant le *pair programming*[1], arrangez-vous pour que la répartition des commits entre vous soit la plus homogène possible.

L'évaluation finale du projet aura lieu la semaine du 21 avril 2025 et la seconde session est prévue la semaine du 19 mai.

Outre l'évaluation du projet chaque membre du groupe sera **interrogé individuellement** lors de l'examen pour valider sa maîtrise des technologies et de l'architecture utilisées dans le cadre du projet. Cet examen permettra de confirmer ou d'invalider la note de projet pour l'étudiant.

Si le projet présenté en première session est jugé trop peu avancé ou trop insuffisant, il n'y aura pas de défense individuelle et les étudiants du groupe seront ajournés en première session.

Si le chargé de cours estime qu'il est possible pour les étudiants de finaliser leur projet dans l'intervalle de temps entre les deux sessions, il le leur signalera. Dans le cas contraire, les étudiants n'auront pas accès à la seconde session.

La note individuelle de chaque étudiant sera fonction de la qualité du projet remis, de sa participation active et régulière et de la qualité de sa défense individuelle.

Si la participation d'un étudiant au projet pendant l'année est jugée insuffisante ou trop peu régulière, l'étudiant sera ajourné en première session et se verra refuser l'accès à la seconde session.

Fraude

Ce travail doit être réalisé en groupes de projet. Chaque groupe produira une réalisation originale qui lui est propre. Toute tentative de réutilisation, même partielle, du travail d'un autre groupe (de la même classe ou d'une autre classe) ou d'une autre personne sera considérée comme de la fraude.

En cas de fraude avérée, les projets concernés recevront une note nulle. Veuillez noter que la fraude et la récidive de fraude peuvent également faire l'objet de sanctions disciplinaires (voir le ROI du Bachelier en Informatique).

---

1. Le *Pair programming* est une méthode de travail dans laquelle deux développeurs travaillent ensemble sur un même poste de travail. La personne qui rédige le code est appelée conducteur (*driver*). La seconde personne, appelée observateur (*observer*), assiste le conducteur en décelant les imperfections, en vérifiant que le code implémente correctement le design et en suggérant des

alternatives de développement. Les rôles s'échangent régulièrement pendant la séance de programmation (Wikipedia). ↩